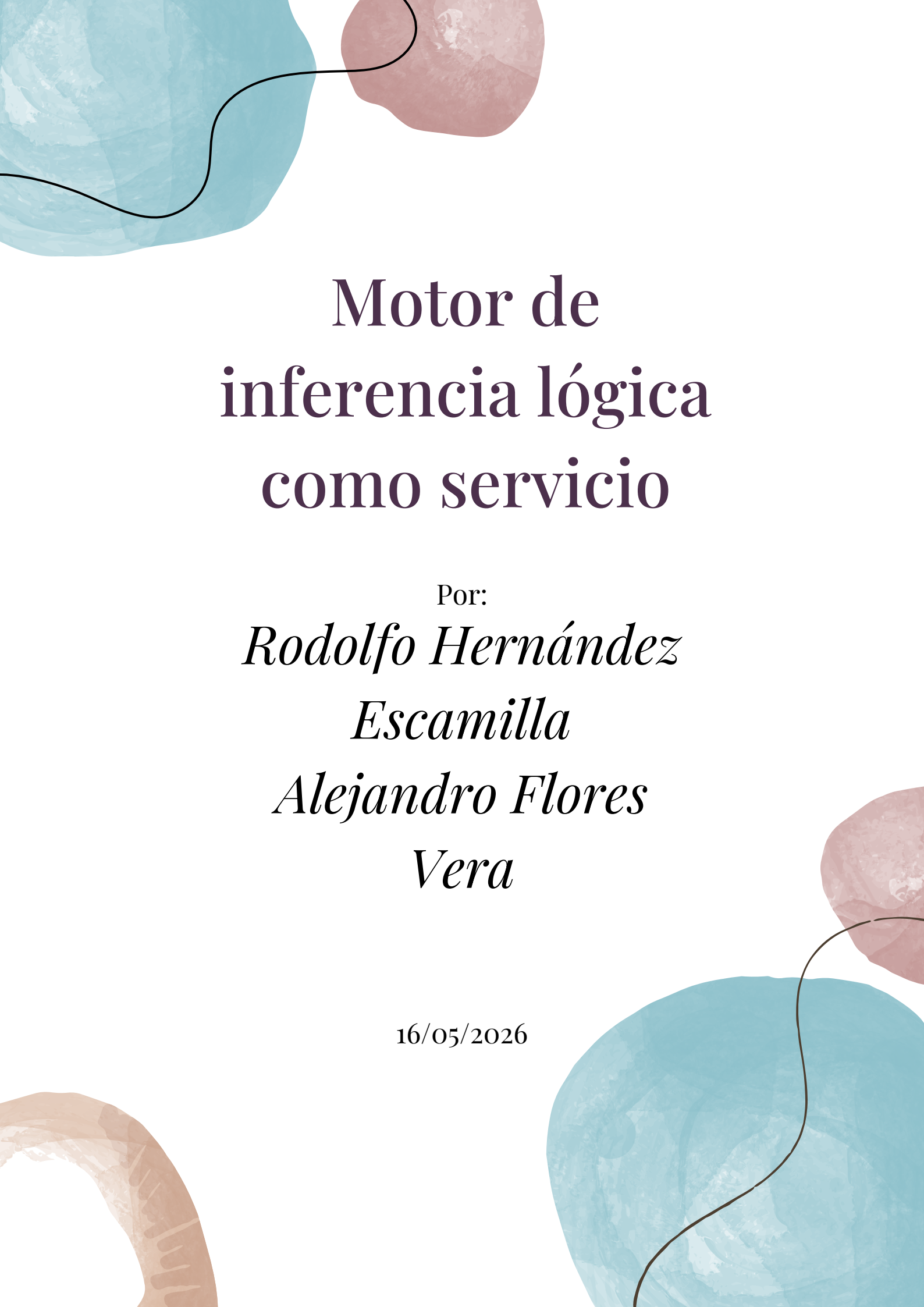




Motor de inferencia lógica como servicio

Por:

Rodolfo Hernández

Escamilla

Alejandro Flores

Vera

16/05/2026

1. Introducción

Este proyecto implementa un Motor de Inferencia Lógica como Servicio, cuyo objetivo es demostrar la coexistencia de múltiples paradigmas de programación dentro de un mismo sistema funcional.

El servicio permite enviar consultas en sintaxis Prolog a un endpoint REST y recibir respuestas derivadas automáticamente por un motor de inferencia simbólica embebido en Node.js.

2. Paradigmas de programación

El funcionamiento correcto del sistema radica en el uso apropiado de tres paradigmas de programación dentro de server.js y base.pl, donde cada uno resuelve una necesidad específica del servicio web.

2.1 Programación lógica (paradigma declarativo)

Implementado en el archivo base.pl a través de Prolog. En lugar de definir algoritmos paso a paso mediante estructuras de control imperativas, el dominio del negocio se modela formalmente mediante:

- **Hechos:** Relaciones estáticas e indexadas como contrato/4, pago_pendiente/2 y tipo_cliente/2.
- **Reglas:** Cláusulas lógicas de Horn (por ejemplo contrato_problemativo/1, elegible_renovacion/1) que utilizan operadores lógicos como la conjunción (.), la disyunción (múltiples definiciones de una regla) y la negación por falla (\+).

El motor de inferencia de Tau Prolog resuelve las consultas del cliente de forma automatizada mediante procesos nativos de unificación de variables y backtracking (búsqueda hacia atrás en el árbol de decisiones).

2.2 Programación Funcional

Usa las capacidades de JavaScript en la sección intermedia del servidor para garantizar la inmutabilidad y predictibilidad en la manipulación de datos mediante funciones puras:

- **normalizeQuery:** Una función pura que toma un string de entrada, remueve espacios y asegura de manera determinista la terminación con punto (.), aislándola de efectos secundarios.
- **formatAnswers y extractBindings:** Transforman las estructuras y árboles de términos nativos devueltos por el motor Prolog en objetos planos JSON listos para el cliente. Utilizan recursión controlada (collectLinks) y mapeos inmutables (.map()) en lugar de ciclos imperativos (for/while) con acumulación de estado global.

2.3 Programación Funcional

Dado que la evaluación lógica y la resolución por backtracking de Tau Prolog son síncronas y consumen tiempo de CPU, se encapsuló todo el ciclo de vida de la consulta dentro de una Promise nativa en la función runPrologQuery.

Al emplear la sintaxis async/await en el endpoint POST /query, el hilo principal de ejecución de Node.js (Event Loop) queda completamente libre mientras el motor procesa las peticiones. Esto evita el bloqueo del servidor y permite despachar múltiples solicitudes HTTP de manera concurrente y no bloqueante.

3. Arquitectura del sistema

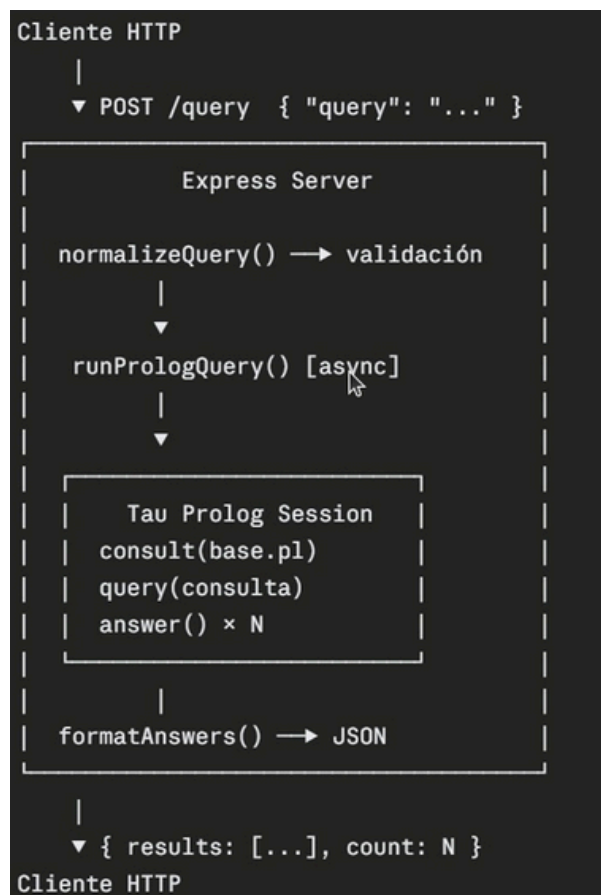
El sistema está diseñado bajo una arquitectura de microservicio monolítico desacoplado por responsabilidades. Toda la lógica de control, la interfaz de red y el motor de inferencia coexisten dentro del mismo proceso de Node.js, lo que elimina la latencia de red interna al interactuar con la base de conocimiento.

A continuación, se detallan los componentes principales y el flujo de datos unificado según la implementación de server.js:

3.1 Componentes del sistema

- **Servidor HTTP y Enrutamiento (Express):** Actúa como la capa externa del servicio. Expone el endpoint de entrada POST /query y el endpoint de monitoreo GET /health. Se encarga de la recepción de payloads en formato JSON, la validación inicial de los tipos de datos y la serialización asíncrona de las respuestas HTTP finales.
- **Capa Funcional de Normalización y Formateo:** Bloque intermedio en JavaScript compuesto por las funciones normalizeQuery, formatAnswers y extractBindings. Su responsabilidad es limpiar la entrada del cliente (asegurando la sintaxis estricta de Prolog con el punto final) y traducir las complejas estructuras del árbol de términos lógicos devueltos por el motor hacia un formato JSON limpio y legible.
- **Motor de Inferencia Lógica (Tau Prolog Core):** Una instancia de la biblioteca tau-prolog configurada con un límite de pasos para prevenir bucles infinitos. Por cada petición recibida, crea una sesión aislada en memoria (pl.create(1000)), analiza gramaticalmente la consulta y computa las soluciones mediante resolución lógica.
- **Base de Conocimiento Estática (base.pl):** Archivo declarativo ubicado en la ruta física ./knowledge/base.pl. Contiene el universo de discurso del sistema, estructurado mediante los hechos e implicaciones lógicas del negocio (auditoría de contratos comerciales).

3.2 Flujo de ejecución de una solicitud



- **Recepción:** El cliente envía una petición POST al endpoint /query con un cuerpo JSON que contiene el string de la consulta lógica.
- **Sanitización:** Express intercepta la petición, valida que el campo query exista y lo transfiere a la capa funcional, la cual normaliza el string de entrada añadiendo el punto (.) si es necesario.
- **Delegación Asíncrona:** El hilo principal invoca a runPrologQuery(normalized), devolviendo una promesa nativa. Dentro de esta, Node.js lee el archivo de la base de conocimiento de forma asíncrona mediante el módulo fs.
- **Evaluación Lógica:** El motor analiza la consulta contra la base de conocimiento cargada. Mediante una llamada recursiva no bloqueante (fetchNext), el motor extrae secuencialmente todas las respuestas válidas encontradas por el mecanismo de backtracking.
- **Serialización:** Las soluciones lógicas crudas son procesadas por formatAnswers para mapear las variables unificadas (bindings). La promesa se resuelve y Express retorna un código de estado HTTP 200 con el arreglo estructurado de resultados.

4. Ejemplo de ejecución

Para validar de forma práctica el funcionamiento del motor de inferencia, se exponen a continuación dos casos de ejecución reales basados exactamente en las reglas y hechos de auditoría comercial definidos en base.pl. Usamos el cliente HTTP del servidor para las pruebas.

4.1 Consulta cerrada (verificación de Regla Booleana)

En este escenario, el cliente desea saber si un recurso específico cumple con una condición lógica, lo que en Prolog se traduce en éxito (true.) o falla (false.).

- **Consulta:** Determinar si el contrato1 es elegible para renovación. Según la base de conocimientos, el contrato1 está activo, pertenece al cliente_a (quien es un usuario premium), pero tiene un pago_pendiente de 5000. Como la regla elegible_renovacion exige que no existan pagos pendientes (\+ pago_pendiente), la inferencia debería fallar.

Petición:

```
curl -X POST http://localhost:3000/query \  
-H "Content-Type: application/json" \  
-d '{"query": "elegible_renovacion(contrato1)"}'
```

Respuesta:

```
{"query": "elegible_renovacion(contrato1).", "results": [], "count": 0, "message": "false."}
```

Al no encontrar ninguna sustitución que satisfaga la regla, el motor retorna un arreglo vacío y el mensaje de Prolog “false”.

4.2 Consulta Abierta con Variables (Extracción de Datos)

En este escenario, se introduce una variable (identificada por iniciar con mayúscula en Prolog: Contrato) para que el mecanismo de backtracking del motor encuentre todas las unificaciones posibles que satisfacen la regla.

- **Consulta:** Identificar cuáles contratos dentro del sistema requieren una auditoría urgente por cumplir con la regla `necesita_auditoria(Contrato)`. Dicha regla evalúa si un contrato está activo y pertenece a un cliente de tipo estandar. Al revisar los hechos, `contrato3` (`cliente_c`, `premium`) y `contrato5` (`cliente_e`, `estandar`) están activos, por lo que solo el `contrato5` debería ser inferido.

Petición:

```
curl -X POST http://localhost:3000/query \  
  -H "Content-Type: application/json" \  
  -d '{"query": "necesita_auditoria(Contrato)"}'
```

Respuesta:

```
{"query": "necesita_auditoria(Contrato).", "results": [{"solution": "Contrato/contrato5", "bindings": {}}], "count": 1}
```

El servidor procesa la consulta de forma asíncrona, extrae la sustitución de variables gracias a la función `extractBindings` y devuelve el resultado de arriba (que solo el `contrato5` cumple con las condiciones de la petición).

5. Conclusiones y extensiones

5.1 Conclusiones

La realización de este proyecto demuestra de forma tangible que los paradigmas de programación no son compartimentos estancos, sino herramientas complementarias dentro de la ingeniería de software moderna. La integración de la lógica declarativa de Prolog con el entorno asíncrono de Node.js elimina la histórica fricción de despliegue de los sistemas expertos tradicionales. Al encapsular el motor Tau Prolog dentro de una API REST con Express, se logró aislar la complejidad de la inferencia lógica (unificación y backtracking) tras una interfaz web estandarizada e intercambiable mediante JSON. Asimismo, la capa funcional demostró ser indispensable para garantizar la predictibilidad y limpieza de los datos antes de interactuar con el núcleo lógico.